Centro Paula Souza

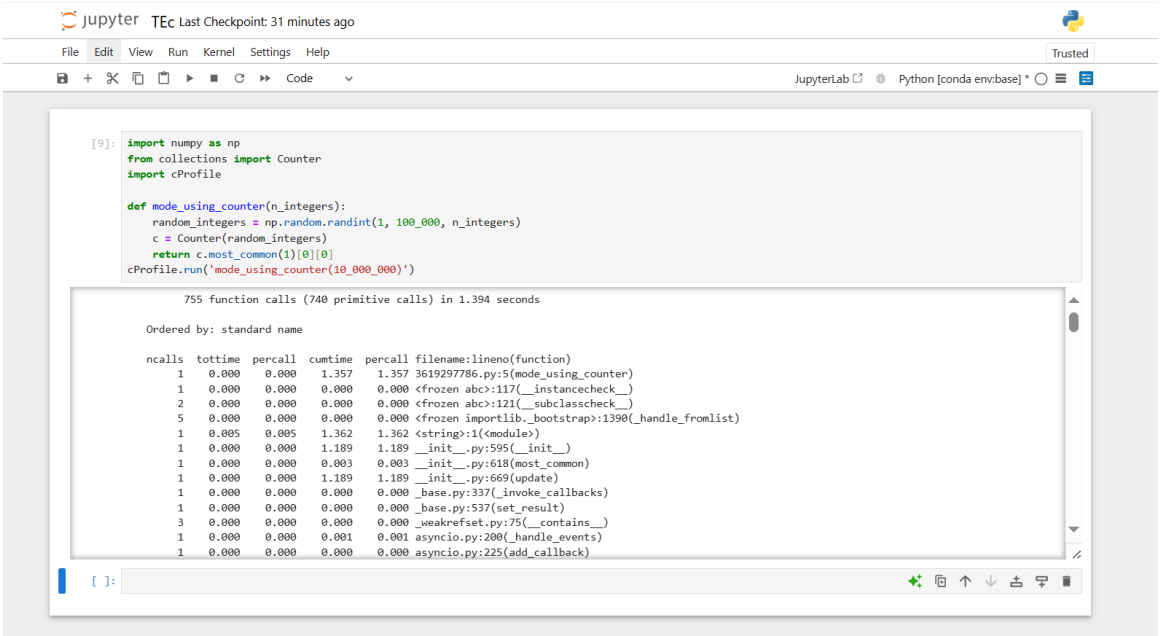
Faculdade de Tecnologia de Votorantim

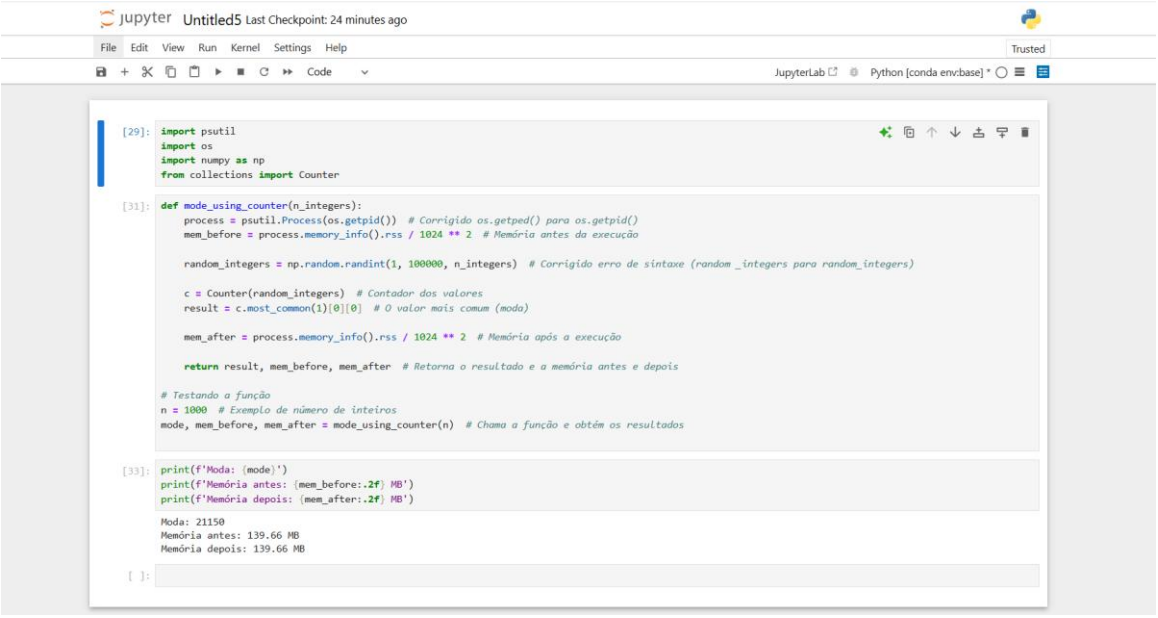
Curso de Ciência de Dados para Negócios

Aluno: Ana Elisa Rubinato Cavalheiro

Técnicas de Programação

A) Memória. Use a função mode_using_counter e calcule o consumo de memória.





B) Função Linear - O(n) - execute a função weighted_mean e para medir o tempo

jupyter Untitled7 Last Checkpoint: 22 minutes ago

File Edit View Run Kernel Settings Help — Trusted

Code — JupyterLab — Python [conda env:base] *

```
[46]: import random
      import time
      import matplotlib.pyplot as plt

[48]: def weighted_mean(list_of_numbers, weights):
          running_total = 0
          for i in range(len(list_of_numbers)):
              running_total += (list_of_numbers[i] * weights[i])
          return running_total / sum(weights)

[50]: def medir_tempo(tamanho):
          numeros = [random.uniform(1, 100) for _ in range(tamanho)]
          pesos = [random.uniform(1, 10) for _ in range(tamanho)]

          inicio = time.time()
          resultado = weighted_mean(numeros, pesos)
          fim = time.time()

          tempo_decorrido = fim - inicio
          return tempo_decorrido

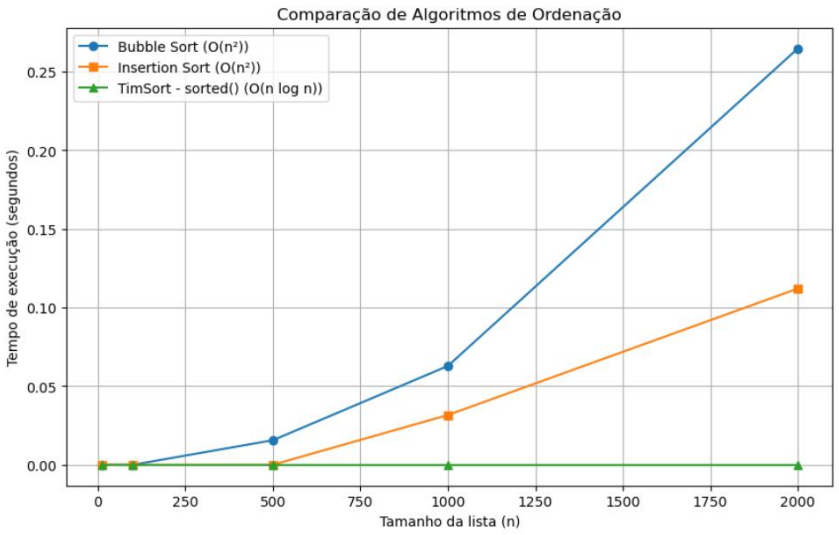
      tamanho_da_lista = 1000
      tempo_execucao = medir_tempo(tamanho_da_lista)
      print(f"Tempo de execução para uma lista de tamanho {tamanho_da_lista}: {tempo_execucao} segundos")
```

```
Tempo de execução para uma lista de tamanho 1000: 0.0 segundos
```

```
[ ]:
```

C) Buble Sort Insertion Sort e timS ort

```
Comparando algoritmos de ordenação...
```



1) **O que você observa nos gráficos em relação ao tempo de execução do Bubble Sort e do Insertion Sort?**

O tempo de execução do Bubble Sort cresce mais rapidamente do que o Insertion Sort conforme o tamanho da lista aumenta. Bubble Sort parece ter um desempenho pior, levando mais tempo para ordenar listas maiores.

2) **Qual algoritmo se comportou melhor conforme o tamanho da lista aumentada? Por quê?**

O Timsort teve o melhor desempenho, seguido pelo Insertion Sort e depoiso Bubble Sort. Isso acontece porque o Timsort tem complexidade O(n log n), enquanto os outros dois têm complexidade $O(n^2)$. Algoritmos com complexidade O(n log n) são mais são mais eficientes para grandes volumes de dados.

3) **O Timsort foi mais rápido? Com base na teoria, qual sua complexidade de tempo e por que isso faz diferença?**

Sim, o **Timsort** foi significativamente mais rápido. Sua complexidade é **O(n log n)** , o que o torna muito mais eficiente para listas grandes. A diferença ocorre porque algoritmos **O(n log n)** fazem menos comparações e trocas do que os algoritmos **O(n^2)** , que desativam muito mais operações conforme o tamanho da entrada aumenta.

4) **Comparando os algoritmos:**

- **Quais algoritmos têm complexidade quadrática (O(n^2))?**
  Classificação por bolha (Bubble Sort) e classificação por inserção (Insertion Sort).

- **Qual tem complexidade log-linear (O(n log n))?**
  Timsort

5) **Execute o teste com uma lista de tamanho 3000 ou 4000 (adicione no vetor tamanhos).**
- **O que acontece com o tempo do Bubble Sort?**

O tempo de execução do Bubble Sort aumentou significativamente, possivelmente tornando a execução inviável em um tempo razoável.

- **Por que isso ocorre?**

Porque o Bubble Sort tem complexidade O(n^2), então dobrar o tamanho da lista pode quadruplicar o tempo de execução. Como resultado, a ordenação se torna muito lenta para listas grandes.